Implementation and Analysis of B-tree Index Structures

Hojun Lee (20211277)
UNIST
South Korea
hojunlee@unist.ac.kr

Abstract

This report presents an implementation and experimental analysis of three tree-based index structures: B-tree, B+tree, and B*tree. The indexes are built over 100,000 student records, where the student ID is used as the search key and the in-memory array position is used as the record identifier (RID). The implementation supports insertion, point search, deletion, and range query workloads. The source code, dataset, and experiment outputs are available at https://github.com/foxisdog/CSE321_Database_Systems/tree/main.

1 Problem Statement

Indexes are a core component of database systems because they reduce the cost of locating records in large datasets. This project studies three balanced multi-way search trees that can be used as database index structures: B-tree, B+tree, and B*tree. The objective is not only to implement their operations, but also to compare how their structural policies affect performance and storage utilization.

The input dataset contains 100,000 student records. Each record has a student ID, name, gender, GPA, height, and weight. The implementation loads all records into an in-memory array. The student ID is used as the index key, and the array position is used as the RID. All three trees are evaluated under the same workloads: insertion of all records, point search over 10,000 random keys, range query over a fixed student ID interval, and deletion of randomly selected records. In this report, the symbol d always denotes the configured tree order, or maximum fan-out. Thus, a node can store at most $d-1$ keys in this implementation. The order d is swept from 3 to 100 to examine how fan-out affects runtime, split frequency, node count, and utilization across the full range of practical orders.

2 Fundamental Index Structure

2.1 Node Layout and Capacity Rules

Table 1 summarizes the node fields used by the three implementations. The common fields are the sorted key vector, child pointers, a parent pointer, and a leaf flag. The main difference is where RIDs are stored. In the B-tree and B*tree, every stored key has an RID in the same node, including internal nodes. In the B+tree, internal nodes contain only separator keys, while leaf nodes contain the authoritative key-RID pairs and a `next` pointer for leaf-level scans.

Table 1 also defines the capacity policy. Since d is the maximum fan-out, a node stores at most $d-1$ keys. The B-tree and B+tree use the usual half-full lower bound. The B*tree uses a stricter lower bound in the implementation, computed from approximately two-thirds of the order, which matches the purpose of keeping nodes more densely occupied.

Table 1: Node layout used in the implementation.

Tree	Internal node	Leaf node	Capacity rule
B-tree	keys, RIDs, children	keys, RIDs	max $d-1$, min $\lceil d/2 \rceil - 1$
B+tree	separator keys, children	keys, RIDs, next	max $d-1$, min $\lceil d/2 \rceil - 1$
B*tree	keys, RIDs, children	keys, RIDs	max $d-1$, min $\lceil 2d/3 \rceil - 1$

2.2 B-tree

B-trees were introduced by Bayer and McCreight for maintaining large ordered indexes on random-access storage [1]. Their main purpose is to keep an index balanced while allowing each node to contain many keys and child pointers. This high fan-out reduces the height of the tree, so search, insertion, and deletion require only a small number of node visits even when the dataset is large. In a database system, this property is useful because each node can be viewed as an index page, and reducing tree height reduces the number of page accesses.

The B-tree implementation uses a node structure containing sorted keys, RIDs, child pointers, a parent pointer, and a leaf flag. In this implementation, both internal and leaf nodes may store key-RID pairs. Therefore, a point search first checks the current node and can terminate at an internal node if the key is found. Algorithm 1 shows the search procedure.

Algorithm 1 B-tree search

```
BTreeSearch(x, k)
if k is stored in node x then
    return the RID associated with k
end if
if x is a leaf then
    return not found
end if
choose child c using the separator keys of x
return BTreeSearch(c, k)
```

If the key is not found and the node is not a leaf, the search follows the child pointer selected by binary search over the separator keys. Algorithm 1 reflects one difference between the B-tree and the B+tree: a B-tree lookup may finish at an internal node. Since internal nodes may store RIDs, the tree does not always need to reach a leaf for point search.

Insertion descends to the target leaf node and inserts the new key-RID pair in sorted order. Algorithm 2 summarizes the key steps.

Algorithm 2 B-tree insertion

```
BTreeInsert(k, RID)
descend to the leaf node x where k belongs
insert (k, RID) into x in sorted order
while x has more than d − 1 keys do
    split x around the median key
    promote the median key-RID pair to the parent
    x ← parent(x)
end while
if the root overflows then
    create a new root
end if
```

If the node overflows, the median key is promoted to the parent. The original node is reused as the left node, a new right node is allocated, and the keys greater than the median are moved to the right node. This split process may propagate upward until the root is reached. If the root overflows, a new root is created. The insertion policy in Algorithm 2 keeps every node within the configured order d. Since a node may contain at most $d-1$ keys, overflow is detected exactly when this bound is exceeded.

For deletion, I referred to the top-down B-tree deletion strategy described in CLRS [3]. The main idea is to avoid descending into a child that cannot safely lose one key. Algorithm 3 summarizes this policy.

Algorithm 3 Top-down B-tree deletion

```
BTreeDelete(x, k)
if k is stored in internal node x then
    replace k by its predecessor or successor
end if
choose the child c that should contain k
if c has the minimum number of keys then
    if an adjacent sibling has an extra key then
        rotate a key through the parent
    else
        merge c with an adjacent sibling
    end if
end if
continue deletion in the selected child
if the root becomes empty then
    promote its only child as the new root
end if
```

Before the algorithm moves into a child with the minimum number of keys, it first borrows a key from a sibling if possible. If neither sibling has an extra key, the child is merged with a sibling before the descent continues. The top-down policy in Algorithm 3 makes the recursive deletion step simpler because the selected subtree is prepared to tolerate the removal of one key. It also makes the implementation easier to reason about: underflow is handled before the recursive descent rather than after returning from it. When deleting a key stored in an internal node, the implementation replaces it with its predecessor or successor and then recursively deletes the replacement key.

2.3 B+tree

The B+tree is implemented to improve range-query behavior [2]. Compared with the B-tree, internal nodes in the B+tree store only separator keys and child pointers. All key-RID pairs are stored in leaf nodes. As a result, point search always descends to a leaf, even if an internal separator key equals the target key. This can make point search less favorable than the B-tree in this implementation, but it creates a much better layout for range scans.

The key motivation of the B+tree is that leaf nodes are connected by linked pointers. Algorithm 4 shows how this design changes range query processing.

Algorithm 4 B+tree range query

```
BPlusRangeSearch(lo, hi)
leaf ← FindLeaf(lo)
while leaf ≠ null do
    for each key in leaf do
        if key > hi then
            return results
        end if
        if key ≥ lo then
            append (key, RID) to results
        end if
    end for
    leaf ← leaf.next
end while
```

A range query first finds the leaf that may contain the lower bound, then scans leaf nodes sequentially through the linked list until the upper bound is passed. This avoids repeated root-to-leaf searches for each key in the range. Algorithm 4 is the reason the B+tree is suitable for analytical queries over key intervals. The tree performs one root-to-leaf traversal to reach the lower bound and then follows the leaf-level linked list. Therefore, range query performance depends more on the number of qualifying leaf entries than on repeated traversal from the root.

The B+tree split policy also differs from the B-tree. Algorithm 5 shows the two cases.

Algorithm 5 B+tree split policy

```
BPlusSplit(node)
if node is a leaf then
    split key-RID pairs into two leaves
    link the right leaf after the left leaf
    copy the first key of the right leaf to parent
else
    split separator keys and child pointers
    push the middle separator key to parent
end if
```

When a leaf node overflows, the key-RID pairs are divided between two leaves, and the first key of the right leaf is copied to the parent as a separator. The copied key remains in the leaf. For internal node splits, the middle separator key is pushed up to the parent and removed from the child node. The split behavior in

Algorithm 5 preserves the B+tree invariant that all RIDs remain in leaves. Thus, the B+tree trades some point-search flexibility for a leaf-level sequential access path.

B+tree deletion first removes the key from a leaf, because leaves are the only nodes that store RIDs. Algorithm 6 shows the underflow handling.

Algorithm 6 B+tree deletion

```
BPlusDelete(k)
leaf ← FindLeaf(k)
remove (k, RID) from leaf
if leaf underflows then
    if a sibling has extra keys then
        borrow one key-RID pair from the sibling
        update the parent separator key
    else
        merge leaf with an adjacent leaf
        repair the leaf next pointer
        remove the obsolete separator from the parent
        recursively fix parent underflow
    end if
end if
if the root becomes empty then
    promote its only child as the new root
end if
```

If the leaf still satisfies the minimum occupancy, deletion ends. Otherwise, the implementation borrows from a sibling and updates the parent separator. If borrowing is impossible, it merges with a sibling, repairs the linked-leaf chain, removes the corresponding separator from the parent, and recursively fixes any parent underflow. Algorithm 6 is more delicate than B-tree deletion because separator keys are copies rather than authoritative records. Therefore, borrowing from the left or right sibling must also refresh the separator stored in the parent. During a merge, the leaf-level linked list must remain connected so that range queries still scan leaves in key order after deletions.

The B-tree and B*tree store RIDs in internal nodes as well as leaves, so search may terminate before reaching a leaf. The B+tree separates navigation from data storage: internal nodes guide the search, while leaves store all key-RID pairs and form a linked list for range scans.

3 Advanced Index Structures (B*tree)

The B*tree keeps the general search semantics of the B-tree but changes the overflow handling policy during insertion [2]. The motivation is to improve node utilization and reduce the number of structural splits. A standard B-tree immediately splits an overflowing node into two nodes. Although this is simple, it can leave nodes only moderately full. The B*tree tries to delay splitting by redistributing keys with a sibling first.

In this implementation, when a non-root node overflows, the algorithm first checks whether the left or right sibling has free space. Algorithm 7 captures this decision process.

Algorithm 7 B*tree overflow handling

```
BStarHandleOverflow(node)
if node is root then
    split the root as in B-tree
else if left or right sibling has space then
    redistribute keys through the parent separator
else
    combine node, sibling, and parent separator
    split the combined keys into three nodes
    place two separator keys in the parent
end if
```

If a sibling has space, keys are redistributed through the parent separator. Only when redistribution is impossible does the implementation perform a 2-to-3 split. In a 2-to-3 split, two full sibling nodes and one parent separator are combined, then divided into three nodes. Two separator keys are installed in the parent. Compared with the B-tree, Algorithm 7 requires more complex insertion logic because it coordinates with siblings and the parent before deciding how to handle overflow. However, this extra work is intended to produce denser nodes. The experiments show this effect clearly: the B*tree consistently achieves higher final node utilization than both the B-tree and the B+tree.

Deletion in the B*tree implementation follows the same top-down structure as B-tree deletion, but it uses the B*tree's stricter minimum occupancy threshold. Algorithm 8 shows the policy.

Algorithm 8 B*tree deletion

```
BStarDelete(x, k)
choose the child c that should contain k
if c has the B* minimum number of keys then
    if an adjacent sibling has extra keys then
        redistribute one key through the parent
    else
        merge c with an adjacent sibling
        remove the separator from the parent
    end if
end if
delete k from the prepared subtree
if the root becomes empty then
    promote its only child as the new root
end if
```

Before descending into a child that is at the minimum, the algorithm first tries to borrow through the parent separator. If neither sibling has spare keys, it merges adjacent nodes and then continues deletion in the merged node. This deletion policy is not the exact inverse of the 2-to-3 insertion split; instead, it preserves the ordered, balanced tree structure while applying the denser B*tree minimum threshold. Algorithm 8 differs from B*tree insertion in its goal. Insertion delays splits to raise utilization, while deletion prevents underflow and preserves balance. Since the tree is already dense after insertion, the stricter minimum occupancy makes borrow-or-merge decisions occur earlier than in the B-tree.

Table 2: Algorithmic comparison.

Tree	Main idea	Strength	Cost
B-tree	balanced multi-way index	simple point search and update	lower utilization than B*tree
B+tree	records only in linked leaves	fast range scan	point search always reaches leaf
B*tree	redistribute before split	high utilization, fewer splits	more complex insertion

3.1 Comparative Discussion

Table 2 compares the three structures. The B-tree is the most direct balanced index in this project: it supports search, insertion, and deletion with a uniform node layout. The B+tree changes the node layout to optimize range scans. Since all records are stored in linked leaves, it can process a range after one lower-bound search. The B*tree changes the insertion policy rather than the search model. By redistributing with siblings before splitting and using 2-to-3 splits, it keeps nodes more densely filled. This explains why the B*tree shows higher utilization in the insertion experiment.

The theoretical reason for the B*tree's higher space utilization is its delayed split policy. A B-tree splits one overflowing node into two nodes immediately, so the resulting nodes can be only moderately full. In contrast, the B*tree first tries to move keys into a sibling through the parent separator. If both nodes are full, the implementation redistributes two nodes plus one parent separator into three nodes. This policy increases the average number of keys per node, which is exactly what appears in the measured utilization values: at $d = 10$, the B*tree reaches 84.48% utilization, compared with 67.80% for the B-tree and 71.49% for the B+tree. The B+tree shows the opposite tradeoff. Its node layout is less favorable for point search, but the linked leaf layer gives a large range-query advantage. Thus, the experimental results match the structural motivation of the three designs rather than only their asymptotic height.

This utilization difference also follows from the split policies. A standard B-tree split turns one overflowing node into two nodes, so the resulting occupancy is roughly one half. A B*tree delays the split until it combines two full nodes and one parent separator into three nodes. The immediate post-split occupancy is approximately

$$\frac{2(d-1)+1}{3(d-1)} = \frac{2d-1}{3d-3} \approx \frac{2}{3},$$

which explains why the B*tree maintains higher utilization after insertion.

4 Experimental Results and Analysis

The experiments were conducted on an Acer Swift Go 16 laptop equipped with a 13th Gen Intel Core i7-13620H processor (16 logical cores) and 16 GB of RAM, running Linux (kernel 7.0.3). The implementation was compiled with GCC 16.1.1 using `g++ -O2 -std=c++17`. All experiments are driven by a single binary built from `experiments/experiment.cc`, which sweeps d from 3 to 100 and writes a summary CSV used to generate the plotting tables in this report. All trees were measured in the same environment using the same 100,000-record dataset. Point-search mean times are measured with `std::chrono::high_resolution_clock` over one batch of 10,000 randomly selected keys, and the returned RIDs are accumulated into a volatile checksum so the compiler cannot remove the search calls. Range query times are the minimum of seven repeated runs to suppress OS scheduling jitter.

4.1 Insertion and Parameter Tuning

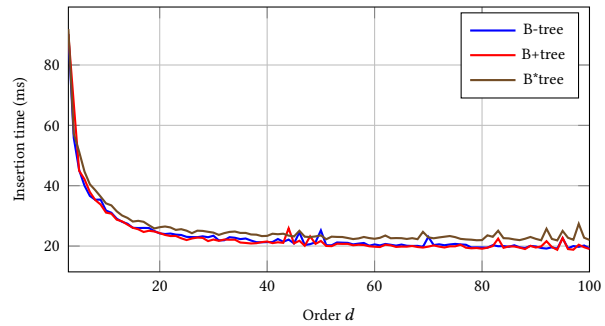

Figure 1: Insertion time for 100,000 records vs. order d.

Figure 1 shows a clear diminishing-return pattern. Moving from $d = 3$ to $d = 10$ cuts insertion time substantially for all three structures because higher fan-out lowers tree height and split frequency. The curves flatten above $d \approx 20$ as per-node binary search and vector-copy costs begin to offset the benefit of reduced height. The three structures are close in insertion time at many orders, but the B*tree often pays extra coordination cost for redistribution and 2-to-3 splits. This cost is traded for fewer splits and higher node utilization (≈84% at $d = 10$ versus ≈68% for B-tree).

Table 3: Structural metrics after inserting all records.

d	Metric	B-tree	B+tree	B*tree
3	splits	74,357	116,779	62,746
3	util.	67.23%	71.39%	79.67%
10	splits	16,381	17,939	13,146
10	util.	67.80%	71.49%	84.48%
100	splits	1,437	1,468	1,198
100	util.	70.15%	69.67%	84.11%

Table 3 shows the structural side of the same parameter sweep. Increasing d sharply reduces split counts because each node can absorb more insertions before overflowing. From $d = 3$ to $d = 100$, split counts fall by about 52× for the B-tree, 80× for the B+tree, and 52× for the B*tree. Utilization behaves differently: the B-tree and B+tree remain around 67–72%, while the B*tree stays near 80–84% because redistribution and 2-to-3 splits keep nodes more densely packed.

The split-count trend follows from the capacity bound. If N keys are stored in nodes with average utilization u, then the number of allocated nodes is approximately

$$n(d) \approx \frac{N}{u(d-1)}.$$

Since each split is the mechanism that creates new nodes during insertion, fewer required nodes imply fewer splits as d grows, assuming utilization does not collapse.

4.2 Point Search

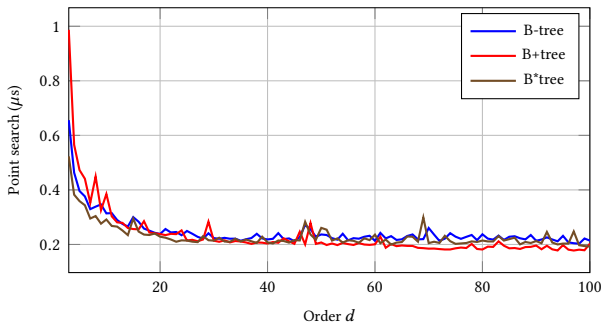

Figure 2: Mean point-search time vs. order d.

Figure 2 shows all three trees operating in a narrow sub-microsecond band throughout the entire d range, with no structure clearly dominant. This result should be interpreted as a warm-memory benchmark. The search queries are executed immediately after insertion and statistics collection, so most tree nodes are already resident in cache. Under this condition, the absolute lookup times are very small.

The relative trend is still meaningful. Search is slower at very small orders because the trees are taller and each lookup visits more nodes. As d increases, the height reduction becomes smaller, and the curves converge. The B+tree is somewhat slower at small d because every successful lookup must reach a leaf, while the B-tree and B*tree can terminate at internal nodes.

4.3 Range Query

The range query computes the average GPA and height of male students whose student IDs fall in [202000000, 202100000] (nine-digit format matching the dataset). The B-tree and B*tree evaluate this by searching every key in the interval; the B+tree finds the lower-bound leaf once and then scans the linked-leaf chain.

Figure 3 shows the largest inter-structure gap of any workload. The B+tree is tens of times faster than the B-tree and B*tree across the sweep. The B-tree and B*tree must issue a separate root-to-leaf search for every ID in the interval; the B+tree pays that cost only once, then follows the leaf-level linked list sequentially. Increasing d helps the B-tree and B*tree by shortening each individual lookup but cannot close the gap, because the number of individual lookups remains proportional to the interval width.

This gap follows directly from the implemented algorithms. Let h be the tree height, W be the integer width of the ID interval, and M be the number of records actually scanned in the range.

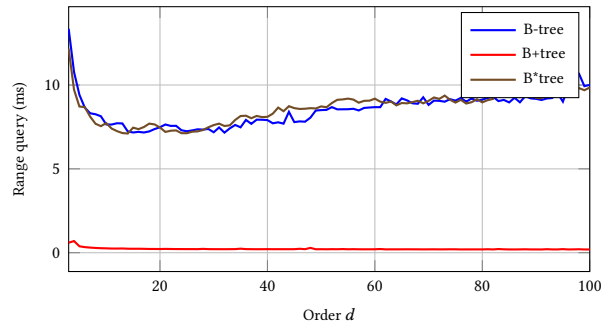

Figure 3: Range-query execution time vs. order d.

The B-tree and B*tree execute one search for every integer key in the interval, giving a cost proportional to $O(Wh)$. The B+tree executes one lower-bound search and then scans linked leaves, giving $O(h + M)$. For the sparse ID interval used here, W is much larger than M, so the linked-leaf scan is expected to be far faster.

4.4 Deletion

Each tree was built with all 100,000 records. Deletion keys were selected by a fixed-seed shuffle; Figure 4 reports the cumulative deletion time at each 10-percentage-point milestone up to 50%.

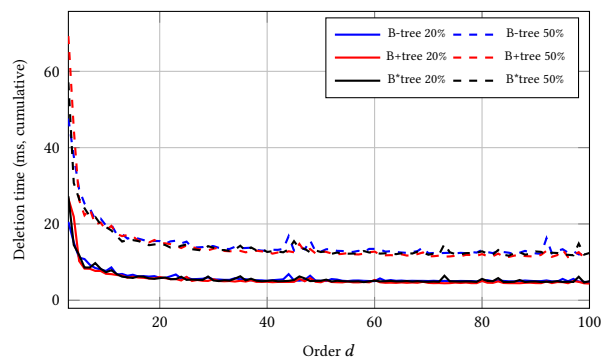

Figure 4: Cumulative deletion time at 20% and 50% ratios vs. order d.

Figure 4 shows three patterns. First, increasing d generally reduces deletion time for all structures. This follows the same high-fan-out effect observed in insertion and search: a larger order reduces tree height, so each top-down deletion visits fewer levels before reaching the target key.

Second, the 50% deletion curve is more than a simple extension of the 20% curve at small and moderate orders. Early deletions are often absorbed by nodes that still have spare keys. After many records have been removed, however, more nodes are close to their minimum occupancy. Later deletions are therefore more likely to require borrowing from a sibling or merging two nodes, which adds extra key movement and parent-separator updates.

Third, the differences among the tree types reflect their structural policies. The B*tree is usually competitive with or faster than the B-tree because its denser initial packing creates fewer nodes

and shorter paths. The B+tree has additional maintenance work: borrowing must refresh separator keys, and merging leaf nodes must also repair the `next` pointer. This overhead is most visible at small orders, while larger orders reduce the number of levels and make all three curves closer.

5 Additional Experiments

The workloads in Section 4 reported execution time as the primary metric. As an additional experiment, I measured the total memory consumed by each tree structure as d varies.

5.1 Memory Usage

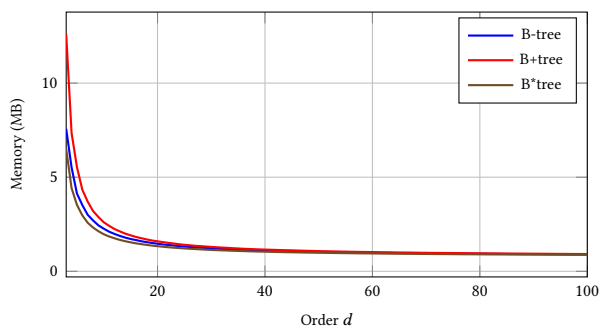

Figure 5: Tree memory usage after inserting all 100,000 records vs. order d.

Memory is estimated by traversing the tree after full insertion and summing, for each node, the struct size (including vector control blocks) plus the actual heap bytes used by the key, RID, and child-pointer vectors.

Figure 5 shows a counterintuitive result: memory *decreases* as d grows. At $d = 3$, the B+tree uses approximately 12.6 MB while the B*tree uses about 6.5 MB; at $d = 100$, all three structures converge near 0.9 MB. The reduction comes from node count dominating per-node size. At small d, the tree is tall and contains many nodes, each small. At large d, far fewer nodes exist, and although each node is larger, the reduction in count outweighs the growth in per-node footprint. The B+tree consistently uses the most memory because storing all RIDs only in leaves forces a larger leaf-level population. The B*tree uses the least at small d because its denser packing produces fewer nodes in total.

The same node-count equation explains the memory result. The amount of indexed key-RID data is fixed across all orders, but the number of allocated nodes is approximately $N/(u(d-1))$. Since each node carries fixed overhead such as a parent pointer, a leaf flag, vector control blocks, and child-pointer storage, larger d reduces the per-node overhead paid by the whole tree. Higher utilization u has the same effect, which is why the B*tree's denser packing also reduces memory usage.

6 Conclusion

The experiments show that the three tree structures have different strengths. The B-tree provides a balanced and direct implementation of point search, insertion, and deletion, with the ability to store RIDs in internal nodes. The B+tree is especially effective for range queries because linked leaves convert an interval lookup into a sequential scan after one lower-bound search. The B*tree improves space utilization and reduces split frequency by redistributing with siblings and using 2-to-3 splits. These results demonstrate that the best index structure depends on the target workload: B+tree is preferable for range-heavy access patterns, while B*tree is attractive when high node utilization and reduced structural overhead are important.

References

[1] Rudolf Bayer and Edward M. McCreight. 1972. Organization and maintenance of large ordered indexes. *Acta Informatica* 1, 3, 173–189. https://doi.org/10.1007/BF00288683

[2] Douglas Comer. 1979. The ubiquitous B-tree. *ACM Computing Surveys* 11, 2 (June 1979), 121–137. https://doi.org/10.1145/356770.356776

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms*, 3rd ed. MIT Press, Cambridge, MA.